



UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO



Tarea 4. Análisis de Expresión Diferencial

Transcriptómica

Estudiante: Pablo Salazar-Méndez

2026-05-16

Table of contents

1	Planteamiento	2
2	Metodología & Justificación	3
2.1	Análisis de Expresión Diferencial	3
2.2	Cargado de paqueterías	3
2.3	Matriz de diseño	4
2.4	Cargado de los datos	5
2.4.1	R	5
2.4.2	Python	6
2.5	Tablas de metadatos	7
2.6	Generación del modelo y visualización inicial	8
2.6.1	DESeq2	8
2.6.2	EdgeR	15
2.7	Contrastes entre factores	19
2.7.1	DESeq2	19
2.7.2	EdgeR	27
2.8	Volcano plot	32
2.8.1	DESeq2	32
2.8.2	EdgeR	35
2.9	Heatmap	35
2.9.1	DESeq2	35
2.9.2	EdgeR	38
2.10	Intersección de genes entre algoritmos y datasets	38
2.10.1	Genes sobre-regulados	39
2.10.2	Genes sub-regulados	40
3	Discusión	41
4	Conclusiones	42
	Referencias	42

1 Planteamiento

Este trabajo compara la detección de genes diferencialmente expresados en datos de RNA-seq procesados con distintos enfoques de cuantificación: alineamiento con **HISAT2** y **STAR**, y pseudoalineamiento con **Salmon**. Para ello, evalúa tanto lecturas **paired-end (PE)** como **single-end (SE)** y aplica tres estrategias de análisis diferencial: **DESeq2 en R**, su implementación en Python mediante **inmoose**, y **edgeR**. A partir de umbrales definidos de ($FDR < 0.05$) y ($|LFC| > 0.5$), analiza la concordancia entre métodos mediante barridos de FDR, volcano plots, heatmaps y diagramas de Venn para genes sobreexpresados y subexpresados.

2 Metodología & Justificación

2.1 Análisis de Expresión Diferencial

En este apartado se compararon los resultados del análisis de expresión diferencial obtenidos con las herramientas `edgeR` [1] y `DESeq2` [2], disponibles en R mediante `Bioconductor`. Adicionalmente, se contrastó la salida de `DESeq2` con la de su implementación en Python, accesible mediante la suite `inmoose` [3].

Es relevante destacar que se siguió el flujo de trabajo descrito en [4] para llevar a cabo este análisis.

2.2 Cargado de paqueterías

Se verificó que los paquetes necesarios estuvieran instalados para su uso; en caso contrario, se instalaron en el ambiente de R, como se muestra a continuación:

```
bioc_packages = c("DESeq2", "edgeR", "ComplexHeatmap")
cran_packages = c(
  "ggplot2", "dplyr", "tibble",
  "patchwork", "VennDiagram", "tidyr",
  "ggVennDiagram"
)

if (!requireNamespace("BiocManager", quietly = TRUE)) {
  install.packages("BiocManager")
}

for (i in 1:2) {
  if (i == 1) {
    packages = bioc_packages
  } else {
    packages = cran_packages
  }

  for (p in packages) {
    # Descargar en caso de que no esté disponible
    if (!requireNamespace(p, quietly = TRUE)) {
      # Elección dinámica del instalador
      if (i == 1) {
        BiocManager::install(p, update = FALSE, ask = FALSE)
      } else {
        install.packages(p)
      }
    }
  }

  invisible(library(p, character.only = TRUE))
}
```

```

}

## Limpieza de variables
rm(bioc_packages, cran_packages, packages, i, p)
invisible(gc())

```

Para el caso de Python se ejecutó un pipeline similar dentro de un ambiente de Conda:

```

conda activate mi_entorno

conda install bioconda::inmooose=0.9.1 \
    numpy=1.26.4 \
    pandas=2.2.3 \
    scikit-learn=1.7.1 \
    matplotlib=3.10.0 \
    seaborn=0.13.2 \
    scipy=1.15.2

```

Una vez hecho eso, se cargó la suite inmooose en el entorno adecuado:

```

import matplotlib.pyplot as plt # Graficar datos
from pathlib import Path # Manejo de archivos
from inmooose.deseq2 import * # Expresión diferencial
from scipy import sparse # Manejo de objetos sparse
import seaborn as sns # Graficar datos
import pandas as pd # Manejo de tablas
import numpy as np # Operaciones vectorizadas
import gc # Recolector de basura

```

2.3 Matriz de diseño

La matriz de diseño, también conocida como matriz modelo, establece el modelo estadístico con el que se describe la relación entre las relaciones en la expresión de los genes, *variable de respuesta*, y aquellas variables que influyen dicha expresión, *variables predictoras* ([5]). En este caso se planteó a la **edad** como **variable predictora**, con la intención de encontrar su influencia en los niveles de expresión sin contemplar otros factores como el sexo ya que todas las muestras provienen de machos ([4]).

El modelo utilizado se muestra a continuación:

$$Y \sim \beta_1 + \beta_2$$

La ausencia de un intercepto, $\beta_1 = 0$, indica que los contrastes se construyeron manualmente, ya que se buscó tener mayor control sobre las variables predictoras, $\beta_2 := \text{edad}$ utilizadas para evaluar la respuesta [4].

2.4 Cargado de los datos

Los datos fueron cargados en objetos únicos dentro de cada lenguaje:

2.4.1 R

```
# ===== CARGANDO DATOS =====

## PAIRED-END
pe_counts = list(
  hisat2 = read.table(
    "results/hisat2_pe/counts_table_hisat2_pe.csv",
    sep="," , row.names=1, header=TRUE
  ),
  star = read.table(
    "results/star_pe/counts_table_star_pe.csv",
    sep="," , row.names=1, header=TRUE
  ),
  salmon = read.table(
    "results/SALMON/PE/salmon_counts_pe.csv",
    sep="," , row.names=1, header=TRUE
  )
)

## SINGLE-END
se_counts = list(
  hisat2 = read.table(
    "results/hisat2_se/counts_table_hisat2_se.csv",
    sep="," , row.names=1, header=TRUE
  ),
  star = read.table(
    "results/star_se/counts_table_star_se.csv",
    sep="," , row.names=1, header=TRUE
  ),
  salmon = read.table(
    "results/SALMON/SE/salmon_counts_se.csv",
    sep="," , row.names=1, header=TRUE
  )
)

# ===== CARGANDO METADATOS =====

## Anotaciones geneid-nombre
geneid_names = read.table(
  "results/support_files/M38_gene_id-name.csv",
  sep="," , row.names = 1, header=TRUE
)
```

```

## Anotaciones geneid-length
geneid_length = list(
  hisat2 = read.table(
    "results/support_files/geneid-length_aligners.csv",
    sep="," , header=TRUE
  ),
  star = read.table(
    "results/support_files/geneid-length_aligners.csv",
    sep="," , header=TRUE
  ),
  salmon = read.table(
    "results/support_files/geneid-length_salmon.csv",
    sep="," , header=TRUE
  )
)

```

2.4.2 Python

Se replicó el procedimiento con adaptaciones al lenguaje (como el uso de diccionarios sobre listas).

```

# ===== CARGANDO DATOS =====

## PAIRED-END
pe_counts = {
  "hisat2" : pd.read_csv(
    "results/hisat2_pe/counts_table_hisat2_pe.csv",
    sep="," , header=0, index_col=0
  ),
  "star" : pd.read_csv(
    "results/star_pe/counts_table_star_pe.csv",
    sep="," , header=0, index_col=0
  ),
  "salmon" : pd.read_csv(
    "results/SALMON/PE/salmon_counts_pe.csv",
    sep="," , header=0, index_col=0
  )
}

## SINGLE-END
se_counts = {
  "hisat2" : pd.read_csv(
    "results/hisat2_se/counts_table_hisat2_se.csv",
    sep="," , header=0, index_col=0
  ),
  "star" : pd.read_csv(
    "results/star_se/counts_table_star_se.csv",

```

```

        sep=",", header=0, index_col=0
    ),
    "salmon" : pd.read_csv(
        "results/SALMON/SE/salmon_counts_se.csv",
        sep=",", header=0, index_col=0
    )
}

# ===== CARGANDO METADATOS =====

## Anotaciones geneid-nombre
geneid_names = pd.read_csv(
    "results/support_files/M38_gene_id-name.csv",
    sep=",", header=0
)

## Quitando el label "Geneid"
for counts_set in (pe_counts, se_counts):
    for key, value in counts_set.items():
        value.index.name = None

## Limpieza de variables
del key, value

```

2.5 Tablas de metadatos

Se contruyeron las tablas de metadatos para cada tabla de conteos, en las que se contienen valores utilizados más tarde para la visualización de los datos [4].

```

# Cada tipo de alineador
count_tools = c("hisat2", "star", "salmon")

# Generación de factores comunes
age = factor(c(rep("a3",3),rep("a18",4)), levels=c("a3","a18"))
sample_color = factor(c(rep("lightblue",3), rep("lightpink",4)))
## Mismo para todos
sample_names = colnames(pe_counts[[count_tools[1]])

# Generación de metadata
metadata = data.frame(sample_names, age) %>%
  remove_rownames %>%
  column_to_rownames(var="sample_names")

rm(sample_names)

```

Para python se siguió un proceso similar.

```

from inmoose.utils import Factor

# Cada tipo de alineador
count_tools = list(pe_counts.keys())

# Generación de factores para la metadata
age = Factor(["a3"] * 3 + ["a18"] * 4)
sample_names = list(pe_counts[count_tools[0]].columns)

metadata = pd.DataFrame(
    {"age" : age}, index=sample_names
)

# Asegurando niveles explícitos
metadata["age"] = Factor(metadata["age"])
metadata["age"] = metadata["age"].cat.reorder_categories(["a3", "a18"])

# Evaluación de la consistencia en nombres
for counts_set in (pe_counts, se_counts):
    for tool, counts in counts_set.items():
        assert all(metadata.index == counts.columns), (
            f"Los nombres de las muestras no coinciden en {tool}"
        )

```

2.6 Generación del modelo y visualización inicial

Cada implementación se ejecutó siguiendo el mismo procedimiento y con sus respectivas adaptaciones, que se desglosaron en dos secciones con una dedicada a las implementaciones de DESeq2.

2.6.1 DESeq2

2.6.1.1 R

```

# ===== DIRECTORIOS =====

results_dir_pe = list(
    hisat2 = "results/hisat2_pe/diffexp_DESeq2",
    star = "results/star_pe/diffexp_DESeq2",
    salmon = "results/SALMON/PE/diffexp_DESeq2"
)
results_dir_se = list(
    hisat2 = "results/hisat2_se/diffexp_DESeq2",
    star = "results/star_se/diffexp_DESeq2",
    salmon = "results/SALMON/SE/diffexp_DESeq2"
)

```

```

# ===== CARGADO DE LOS OBJETOS =====

## PAIRED-END
pe_dds = list()
pe_vsd = list()

## SINGLE-END
se_dds = list()
se_vsd = list()

## Matriz de diseño para todas las tablas
design = model.matrix(~ 0 + age)

# Modelo para todas las tablas
for (i in 1:2) {
  if (i == 1) {
    dds = pe_dds
    counts = pe_counts
    vsd = pe_vsd
    results_dir = results_dir_pe
  } else {
    dds = se_dds
    counts = se_counts
    vsd = se_vsd
    results_dir = results_dir_se
  }

  for (tool in count_tools) {
    # Inicilización del objeto
    tmp_dds = DESeqDataSetFromMatrix(
      countData = round(counts[[tool]]),
      colData = metadata,
      design = ~ 0 + age
    )

    # Filtrado de genes con baja expresión
    keep = filterByExpr(tmp_dds, design)
    tmp_dds = tmp_dds[keep,]
    rm(keep)

    # Transformación estabilizadora de la varianza
    vsd[[tool]] = vst(tmp_dds)
    dds[[tool]] = tmp_dds

    # Guardado de la matriz VSD
    if (!dir.exists(results_dir[[tool]])) {
      dir.create(results_dir[[tool]], recursive = TRUE)
    }
  }
}

```

```

        write.table(
          assay(vsd[[tool]]),
          file = file.path(
            results_dir[[tool]],
            "deseq2-vsd-matrix.csv"
          ),
          sep = ",",
          quote = FALSE,
          row.names = TRUE
        )
      }

      # Guardado final
      if (i == 1) {
        pe_dds = dds
        pe_vsd = vsd
      } else {
        se_dds = dds
        se_vsd = vsd
      }
    }

    # Limpieza de variables
    rm(dds, vsd, tmp_dds, i, tool, counts, results_dir)
    invisible(gc())

```

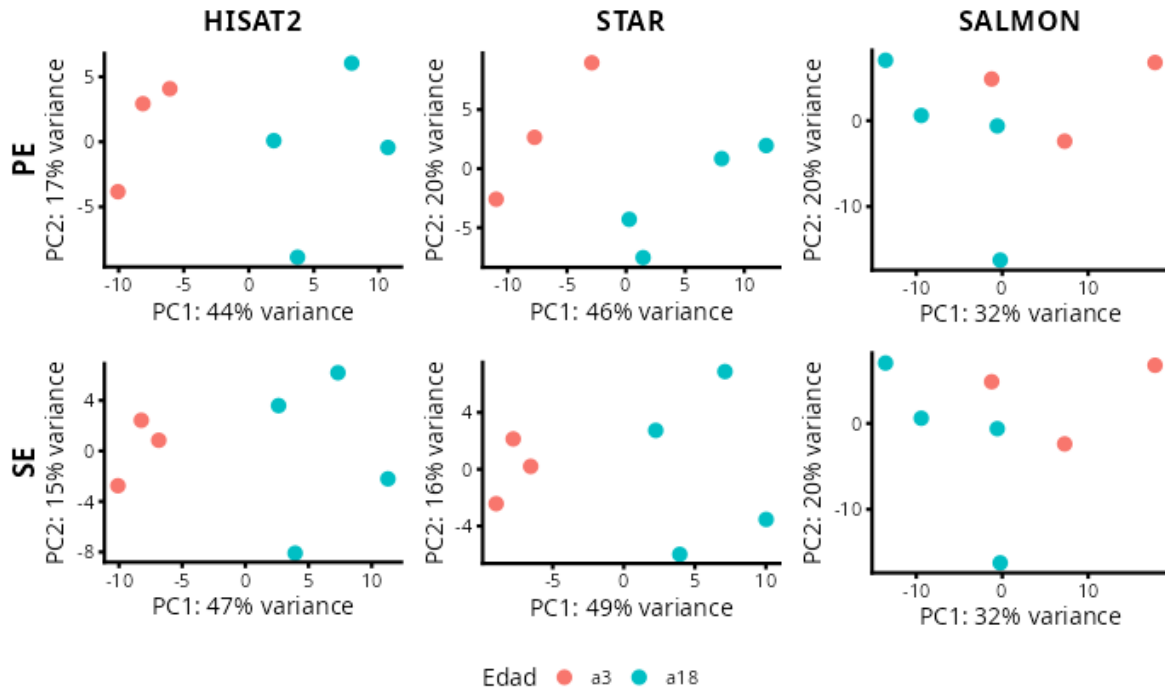


Figura 1: PCA para cada tipo de lectura (PE/SE) y los tres tipos de alineador (HISAT2, STAR y SALMON) tras su procesamiento en DESeq2.

Al no tener un factor como el sexo para generar efecto de lote, la diferencia en las varianzas (hasta este punto) parece deberse al tipo de alineador antes que al tipo de lectura (ver **Figura 1**).

Dicho eso se procedió a ajustar el modelo, encontrar los factores de normalización y de varianza:

```
for (i in 1:2) {
  if (i == 1) {
    dds_objs = pe_dds
  } else {
    dds_objs = se_dds
  }

  # Factores de normalización
  for (tool in count_tools) {
    tmp_dds = dds_objs[[tool]]
    dds_objs[[tool]] = DESeq(tmp_dds)
  }

  # Guardado final
  if (i == 1) {
    pe_dds = dds_objs
  } else {

```

```

        se_dds = dds_objs
    }

    rm(tmp_dds, dds_objs, tool, i)
    invisible(gc())
}

```

2.6.1.2 Python

```

# ===== DIRECTORIOS =====

results_dir_pe = {
    "hisat2": Path("results/hisat2_pe/diffexp_Inmoose"),
    "star": Path("results/star_pe/diffexp_Inmoose"),
    "salmon": Path("results/SALMON/PE/diffexp_Inmoose")
}
results_dir_se = {
    "hisat2": Path("results/hisat2_se/diffexp_Inmoose"),
    "star": Path("results/star_se/diffexp_Inmoose"),
    "salmon": Path("results/SALMON/SE/diffexp_Inmoose")
}

# ===== CARGADO DE LOS OBJETOS =====

## PAIRED-END
pe_dds = {}
pe_vsd = {}

## SINGLE-END
se_dds = {}
se_vsd = {}

## Modelo para todas las tablas
for read_type in ("PE", "SE"):
    if read_type == "PE":
        dds_objs = pe_dds
        counts_set = pe_counts
        vsd_objs = pe_vsd
        results_dir = results_dir_pe
    else:
        dds_objs = se_dds
        counts_set = se_counts
        vsd_objs = se_vsd
        results_dir = results_dir_se

    for tool in count_tools:
        # Matriz de conteos

```

```

counts = counts_set[tool].round().astype(int)

# Metadata en el mismo orden que las columnas de los conteos
metadata_tmp = metadata.loc[counts.columns].copy()

# Inicialización de los objetos
tmp_dds = DESeqDataSet(
    countData=counts.T, # muestras x genes
    clinicalData=metadata_tmp,
    design=~ age"
)

# Filtrado de genes con baja expresión
# Al no haber FilterByExpr() se hará bajo un criterio arbitrario
keep = tmp_dds.counts().sum(axis=0) >= 10
tmp_dds = tmp_dds[:, keep]
del keep

# Transformación estabilizadora de la varianza
tmp_vsd = varianceStabilizingTransformation(tmp_dds)

# Guardado de resultados
vsd_objs[tool] = tmp_vsd
dds_objs[tool] = tmp_dds

# Guardado de la matriz VSD
results_dir[tool].mkdir(parents=True, exist_ok=True)

# Extraer matriz VSD
vsd_values = tmp_vsd.X

# Por si fuera matriz sparse
if sparse.issparse(vsd_values):
    vsd_values = vsd_values.toarray()

# Construir DataFrame: muestras x genes
vsd_df = pd.DataFrame(
    vsd_values,
    index=tmp_vsd.obs_names,
    columns=tmp_vsd.var_names
)

# Guardar como genes x muestras, equivalente a assay(vsd) en R
vsd_df.T.to_csv(
    results_dir[tool] / "inmoose-vsd-matrix.csv",
    index_label="Geneid"
)

# Guardado final

```

```

if read_type == "PE":
    pe_dds = dds_objs
    pe_vsd = vsd_objs
else:
    se_dds = dds_objs
    se_vsd = vsd_objs

del tmp_vsd, tmp_dds, tool, counts_set, results_dir

del read_type, dds_objs, vsd_objs
gc.collect()

```

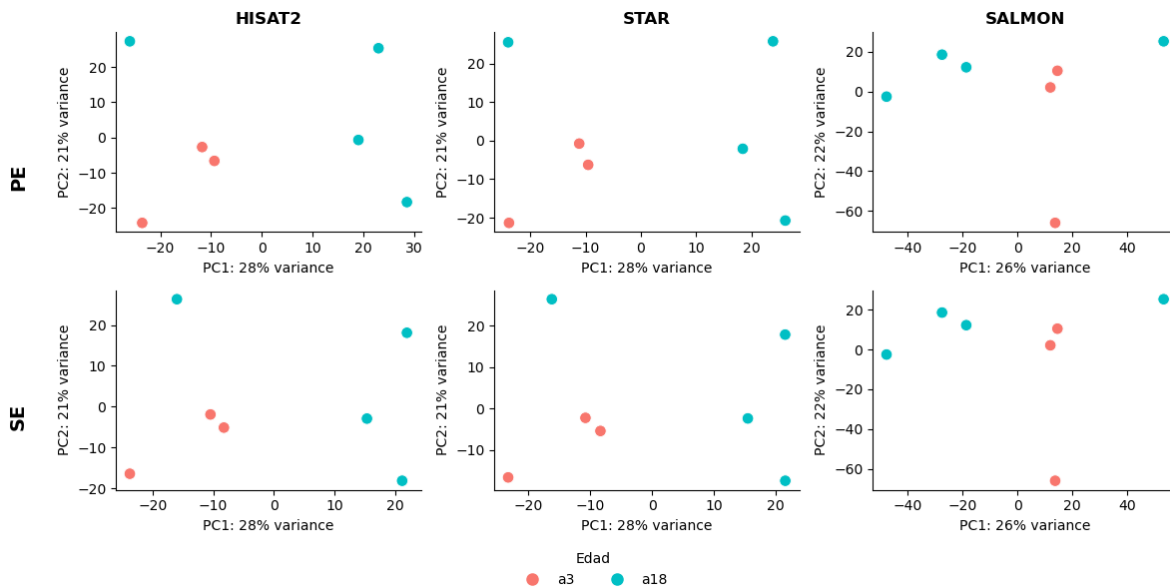


Figura 2: PCA para cada tipo de lectura (PE/SE) y los tres tipos de alineador (HISAT2, STAR y SALMON) tras su procesamiento en DESeq2 en Python.

Se observó que, en cuanto a las varianzas, las distribuciones entre ambas implementaciones son muy similares (ver **Figuras 1 y 2**). Aunque el acomodo espacial pueda lucir desconcertante, de usar procrustes ortogonales para rotar las proyecciones obtenidas por el PCA se podría visualizar un empalme en la mayoría de los alineadores con excepción de SALMON [6].

Sin embargo, la mayor diferencia entre ambos se ve en el porcentaje de varianza que los primeros dos componentes capturan en *inmooose* con respecto a DESeq2.

```

# ===== AJUSTE DEL MODELO =====

for read_type in ["PE", "SE"]:

```

```

if read_type == "PE":
    dds_objs = pe_dds
else:
    dds_objs = se_dds

# La suite inmoose permite la extracción de contrastes
# más adelante en el pipeline
for tool in count_tools:
    # Ajuste del modelo GLM
    dds_objs[tool] = DESeq(dds_objs[tool])

if read_type == "PE":
    pe_dds = dds_objs
else:
    se_dds = dds_objs

```

2.6.2 EdgeR

```

# ===== DIRECTORIOS =====

results_dir_pe = list(
    hisat2 = "results/hisat2_pe/diffexp_DESeq2",
    star = "results/star_pe/diffexp_DESeq2",
    salmon = "results/SALMON/PE/diffexp_DESeq2"
)
results_dir_se = list(
    hisat2 = "results/hisat2_se/diffexp_DESeq2",
    star = "results/star_se/diffexp_DESeq2",
    salmon = "results/SALMON/SE/diffexp_DESeq2"
)

# ===== CARGADO DE LOS OBJETOS edgeR =====

pe_dge = list()
pe_logcpm = list()

se_dge = list()
se_logcpm = list()

for (i in 1:2) {

    if (i == 1) {
        dge = pe_dge
        counts = pe_counts
        logcpm = pe_logcpm
        results_dir = results_dir_pe
    }
}

```

```

} else {
  dge = se_dge
  counts = se_counts
  logcpm = se_logcpm
  results_dir = results_dir_se
}

for (tool in count_tools) {

  # Reordenar metadata según las columnas de la matriz de conteos
  metadata_tool = metadata[colnames(counts[[tool]]), , drop = FALSE]

  stopifnot(all(colnames(counts[[tool]]) == rownames(metadata_tool)))

  # Asegurar que age sea factor
  metadata_tool$age = factor(metadata_tool$age)

  # Diseño específico para esa matriz
  design_tool = model.matrix(~ 0 + age, data = metadata_tool)

  # Inicialización del objeto edgeR
  tmp_dge = DGEList(
    counts = round(counts[[tool]]),
    samples = metadata_tool,
    group = metadata_tool$age
  )

  # Filtrado de baja expresión
  keep = filterByExpr(tmp_dge, design = design_tool)
  tmp_dge = tmp_dge[keep, , keep.lib.sizes = FALSE]

  # Normalización TMM
  tmp_dge = calcNormFactors(tmp_dge)

  # Matriz logCPM para PCA / heatmaps
  logcpm[[tool]] = cpm(tmp_dge, log = TRUE)

  # Guardado de la matriz logCPM
  if (!dir.exists(results_dir[[tool]])) {
    dir.create(results_dir[[tool]], recursive = TRUE)
  }

  write.table(
    logcpm[[tool]],
    file = file.path(
      results_dir[[tool]],
      "edgeR-logCPM-matrix.csv"
    ),
    sep = ",",

```

```

    quote = FALSE,
    row.names = TRUE
  )

  # Guardar también el diseño, porque corresponde a ese objeto
  dge[[tool]] = list(
    dge = tmp_dge,
    design = design_tool
  )
}

if (i == 1) {
  pe_dge = dge
  pe_logcpm = logcpm
} else {
  se_dge = dge
  se_logcpm = logcpm
}
}

rm(
  dge,
  logcpm,
  tmp_dge,
  counts,
  keep,
  metadata_tool,
  design_tool,
  i, tool,
  results_dir
)
invisible(gc())

```

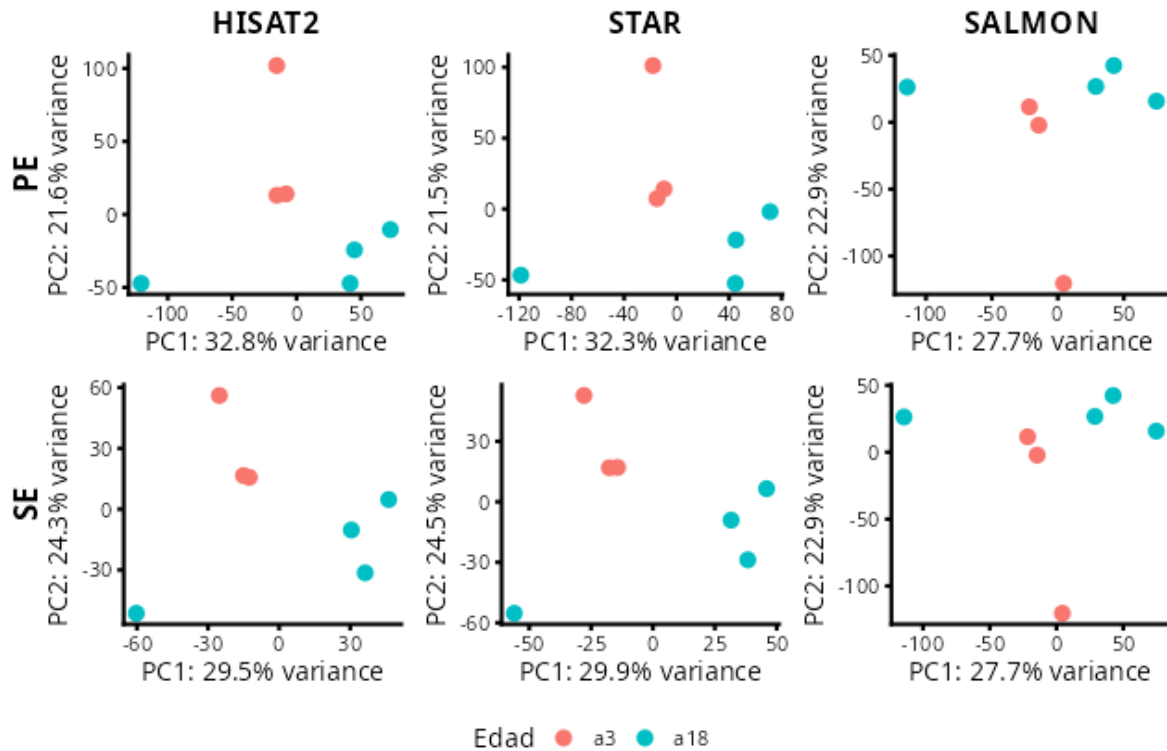


Figura 3: PCA para cada tipo de lectura (PE/SE) y cada tipo de alineador (HISAT2, STAR y SALMON) a partir de su procesamiento por EdgeR.

EdgeR es una herramienta que separa en dos funciones lo que el algoritmo DESeq2 tiene en una sola: 1) la estimación de la dispersión poblacional a partir de las réplicas y 2) el ajuste del modelo GLM. Por ello es que se codificó de tal manera que ambos procesos quedaran incluidos en el pipeline.

```
for (i in 1:2) {
  if (i == 1) {
    dge_objs = pe_dge
  } else {
    dge_objs = se_dge
  }

  for (tool in count_tools) {
    tmp_dge = dge_objs[[tool]]$dge
    design_tool = dge_objs[[tool]]$design

    tmp_dge = estimateDisp(tmp_dge, design_tool)
  }
}
```

```

fit = glmQLFit(
  tmp_dge,
  design_tool,
  robust = TRUE
)

dge_objs[[tool]] = list(
  dge = tmp_dge,
  design = design_tool,
  fit = fit
)
}

if (i == 1) {
  pe_edger_fit = dge_objs
} else {
  se_edger_fit = dge_objs
}
}

rm(tmp_dge, fit, dge_objs, design_tool, tool, i)
invisible(gc())

```

2.7 Contrastes entre factores

Los contrastes se realizaron entre las edades de 18 y 3 meses aprovechando que no se usó una base default.

2.7.1 DESeq2

2.7.1.1 R

```

# ===== CONTRASTES =====

## PE/SE
deseq2_contrasts = makeContrasts(
  a18_vs_a3 = agea18 - agea3,
  levels=design
)

contrast_var = "a18_vs_a3"
invisible(gc())

```

Se contrastó con el mismo diseño para todos los datasets, por lo que las pruebas fueron consistentes tanto en interpretación biológica como en dirección de la prueba estadística.

```

# ===== PRUEBA ESTADÍSTICA =====

# Guardado de los resultados
results_pe = list()
results_se = list()

# Valores de LFC
LFC_unique = 0.5

# Barrido de FDRs
FDR_values = seq(
  from=1e-10,
  to=0.1,
  by=1e-05
)
n = length(FDR_values)

# ===== NO. GENES DIF. EXP =====

deseq2_num_genes_pe = data.frame(
  read_type = rep("PE", n),
  FDR = FDR_values,
  hisat2 = integer(n),
  star = integer(n),
  salmon = integer(n)
)
deseq2_num_genes_se = data.frame(
  read_type = rep("SE", n),
  FDR = FDR_values,
  hisat2 = integer(n),
  star = integer(n),
  salmon = integer(n)
)

for (i in 1:2) {
  if (i == 1) {
    dds = pe_dds
    res = results_pe
    deseq2_num_genes = deseq2_num_genes_pe
  } else {
    dds = se_dds
    res = results_se
    deseq2_num_genes = deseq2_num_genes_se
  }
  for (tool in count_tools) {

    # Contrastado de factores
    res[[tool]] = results(
      dds[[tool]],

```

```

        contrast=deseq2_contrasts[,contrast_var]
    )

    # Nombre de los genes
    res[[tool]]$Gene_name = geneid_names[
        rownames(res[[tool]]),
    ]

    # Número de genes con FDR variable y LFC fijo
    j = 1
    for (fdr in FDR_values) {
        up = (res[[tool]]$log2FoldChange > LFC_unique) &
            (res[[tool]]$padj < fdr)
        up[which(is.na(up))] = FALSE

        down = (res[[tool]]$log2FoldChange < -LFC_unique) &
            (res[[tool]]$padj < fdr)
        down[which(is.na(down))] = FALSE

        # Genes diferencialmente expresados
        deseq2_num_genes[[tool]][j] = sum(up) + sum(down)

        j = j + 1
    }
}

# Guardado de resultados
if (i == 1) {
    pe_dds = dds
    results_pe = res
    deseq2_num_genes_pe = deseq2_num_genes
} else {
    se_dds = dds
    results_se = res
    deseq2_num_genes_se = deseq2_num_genes
}
}

# Limpieza de variables
rm(
    dds,
    res,
    deseq2_num_genes,
    tool,
    i, j
)
invisible(gc())

```

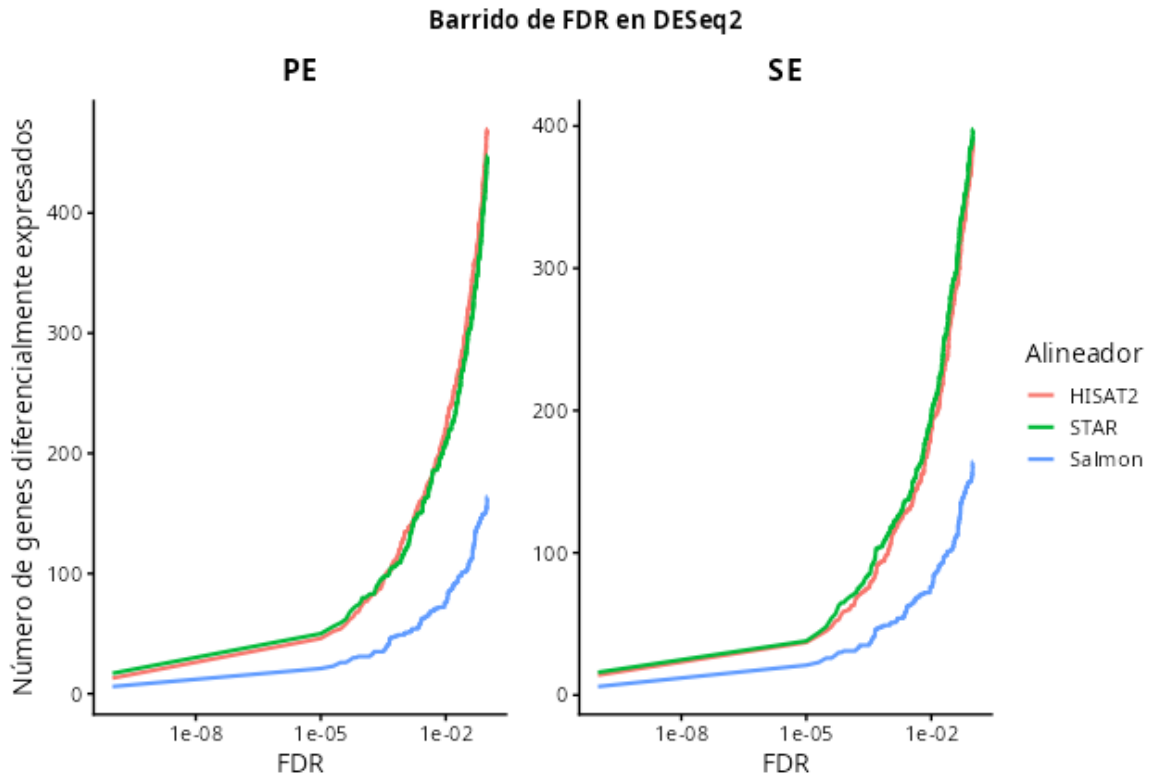


Figura 4: Barrido de valores de FDR y su efecto en el número de genes diferencialmente expresados para el método DESeq2.

En la **Figura 4** se observó que tanto HISAT2 como STAR son consistentes en el número de genes diferencialmente expresados en el barrido de valores de FDR, en contraste con Salmon que de forma general se mantuvo con números bajos de estos genes. Con estos resultados se optó por un FDR laxo de 0.05, es decir, se aceptaron cinco falsos positivos por cada cien genes.

```
# ===== TABLAS =====

# Directorios
results_dir_pe = list(
  hisat2 = "results/hisat2_pe/diffexp_DESeq2",
  star = "results/star_pe/diffexp_DESeq2",
  salmon = "results/SALMON/PE/diffexp_DESeq2"
)
results_dir_se = list(
  hisat2 = "results/hisat2_se/diffexp_DESeq2",
  star = "results/star_se/diffexp_DESeq2",
  salmon = "results/SALMON/SE/diffexp_DESeq2"
)
```

```

# Parámetros
FDR_unique = 0.05

for (i in 1:2) {
  if (i == 1) {
    results_dir = results_dir_pe
    res = results_pe
  } else {
    results_dir = results_dir_se
    res = results_se
  }

  for (tool in count_tools) {

    # Creación del directorio en caso de no existir
    if (!dir.exists(results_dir[[tool]])){
      dir.create(results_dir[[tool]], recursive = TRUE)
    }

    # ===== Tablas con FDR y LFC fijos =====

    up = (res[[tool]]$log2FoldChange > LFC_unique) &
      (res[[tool]]$padj < FDR_unique)
    up[which(is.na(up))] = FALSE

    down = (res[[tool]]$log2FoldChange < -LFC_unique) &
      (res[[tool]]$padj < FDR_unique)
    down[which(is.na(down))] = FALSE

    # Guardado de tablas UP/DOWN
    write.table(
      res[[tool]][up,],
      file.path(
        results_dir[[tool]],
        paste("deseq2-DEG-UP-", FDR_unique, ".csv", sep="")
      ),
      sep = ",", quote = FALSE, row.names = T
    )
    write.table(
      res[[tool]][down,],
      file.path(
        results_dir[[tool]],
        paste("deseq2-DEG-DOWN-", FDR_unique, ".csv", sep="")
      ),
      sep = ",", quote = FALSE, row.names = T
    )
  }
}

```

```
rm(res, results_dir, tool, i, up, down)
invisible(gc())
```

2.7.1.2 Python

```
# ===== CONTRASTES =====

pe_res = {}
se_res = {}

for read_type in ["PE", "SE"]:

    if read_type == "PE":
        dds_objs = pe_dds
        res_objs = pe_res
    else:
        dds_objs = se_dds
        res_objs = se_res

    for tool in count_tools:

        # Contrastes con numerador - denominador
        # numerador = 18 meses
        # denominador = 3 meses
        res = dds_objs[tool].results(
            contrast=["age", "a18", "a3"]
        )

        res_objs[tool] = res

    # Guardado de resultados
    if read_type == "PE":
        pe_res = res_objs
    else:
        se_res = res_objs
```

Los contrastes siguieron la misma interpretación biológica que en R y misma dirección estadística.

```
# Parámetros
LFC_unique = 0.5

FDR_values = np.arange(
    1e-10,
    0.1,
    1e-5
)
```

```

n = len(FDR_values)

# Número de genes dif. exp.
inmoose_num_genes_pe = pd.DataFrame({
    "read_type": ["PE"] * n,
    "FDR": FDR_values,
    "hisat2": np.zeros(n, dtype=int),
    "star": np.zeros(n, dtype=int),
    "salmon": np.zeros(n, dtype=int)
})

inmoose_num_genes_se = pd.DataFrame({
    "read_type": ["SE"] * n,
    "FDR": FDR_values,
    "hisat2": np.zeros(n, dtype=int),
    "star": np.zeros(n, dtype=int),
    "salmon": np.zeros(n, dtype=int)
})

# Mapeo de genes
gene_name_map = geneid_names.set_index("Geneid")["Gene_name"]

for read_type in ["PE", "SE"]:

    if read_type == "PE":
        res_objs = pe_res
        num_genes_df = inmoose_num_genes_pe
    else:
        res_objs = se_res
        num_genes_df = inmoose_num_genes_se

    for tool in count_tools:

        # Resultado del contraste
        res = res_objs[tool].copy()

        # Agregar nombre de gen
        res["Gene_name"] = res.index.map(gene_name_map)

        # Número de genes con FDR variable y LFC fijo
        for j, fdr in enumerate(FDR_values):

            up = (
                (res["log2FoldChange"] > LFC_unique) &
                (res["adj_pvalue"] < fdr)
            )
            up = up.fillna(False)

```

```

down = (
  (res["log2FoldChange"] < -LFC_unique) &
  (res["adj_pvalue"] < fdr)
)
down = down.fillna(False)

num_genes_df.loc[j, tool] = up.sum() + down.sum()

# Guardar el resultado actualizado con Gene_name
res_objs[tool] = res

# Guardado final
if read_type == "PE":
  pe_res = res_objs
  inmoose_num_genes_pe = num_genes_df
else:
  se_res = res_objs
  inmoose_num_genes_se = num_genes_df

del res_objs, num_genes_df, tool

```

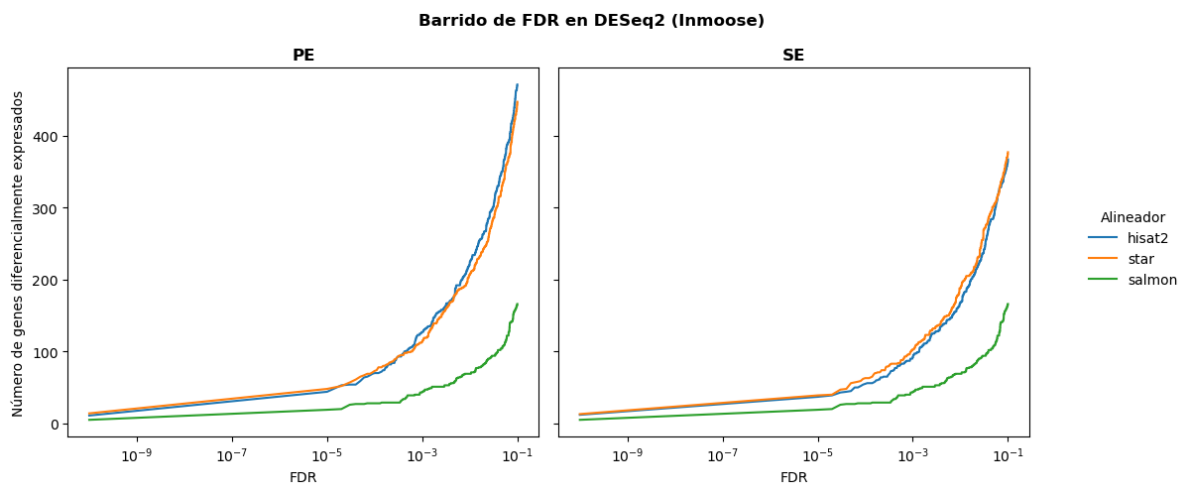


Figura 5: Barrido de valores de FDR y su efecto en el número de genes diferencialmente expresados para el método DESeq2 implementado en inmoose.

La **Figura 5** mostró una tendencia similar a la vista en la implementación en R de este algoritmo, por lo que se utilizó el mismo $FDR < 0.05$.

2.7.2 EdgeR

En el caso de EdgeR los contrastes se establecieron a la par que se aplicó la prueba estadística, manteniendo la dirección de la misma para mantener consistencia en los resultados.

```
# ===== CONTRASTE =====

# Resultados de la prueba
edger_results_pe = list()
edger_results_se = list()

# Parámetros
LFC_unique = 0.5

# ===== BARRIDO FDR =====

# FDR para el testing
FDR_values = seq(
  from = 1e-10,
  to = 0.1,
  by = 1e-05
)

n = length(FDR_values)

# Dataframes para guardar número de genes DEG por FDR
edger_num_genes_pe = data.frame(
  read_type = rep("PE", n),
  FDR = FDR_values,
  hisat2 = integer(n),
  star = integer(n),
  salmon = integer(n)
)

edger_num_genes_se = data.frame(
  read_type = rep("SE", n),
  FDR = FDR_values,
  hisat2 = integer(n),
  star = integer(n),
  salmon = integer(n)
)

# Contraste edgeR
design = model.matrix(~ 0 + age, data = metadata)

contrast_var = "a18_vs_a3"

for (i in 1:2) {
```

```

if (i == 1) {
  fit_objs = pe_edger_fit
  res = edger_results_pe
  edger_num_genes = edger_num_genes_pe
} else {
  fit_objs = se_edger_fit
  res = edger_results_se
  edger_num_genes = edger_num_genes_se
}

for (tool in count_tools) {

  design_tool = fit_objs[[tool]]$design

  edger_contrast = makeContrasts(
    a18_vs_a3 = agea18 - agea3,
    levels = design
  )

  # Prueba del contraste con quasi-likelihood
  qlf = glmQLFTest(
    fit_objs[[tool]]$fit,
    contrast = edger_contrast[, contrast_var]
  )

  # Tabla completa de resultados
  res[[tool]] = topTags(
    qlf,
    n = Inf,
    sort.by = "PValue"
  )$table

  # Nombre de los genes
  res[[tool]]$Gene_name = geneid_names[
    rownames(res[[tool]]),
  ]

  # ===== Barrido de FDR con LFC fijo =====

  j = 1

  for (fdr in FDR_values) {

    up = (res[[tool]]$logFC > LFC_unique) &
      (res[[tool]]$FDR < fdr)
    up[is.na(up)] = FALSE

    down = (res[[tool]]$logFC < -LFC_unique) &
      (res[[tool]]$FDR < fdr)
  }
}

```

```

    down[is.na(down)] = FALSE

    # Genes diferencialmente expresados
    edger_num_genes[[tool]][j] = sum(up) + sum(down)

    j = j + 1
  }
}

# Guardado de resultados
if (i == 1) {
  edger_results_pe = res
  edger_num_genes_pe = edger_num_genes
} else {
  edger_results_se = res
  edger_num_genes_se = edger_num_genes
}
}

# Limpieza
rm(
  fit_objs,
  res,
  edger_num_genes,
  tool,
  i,
  j,
  qlf,
  design
)

invisible(gc())

```

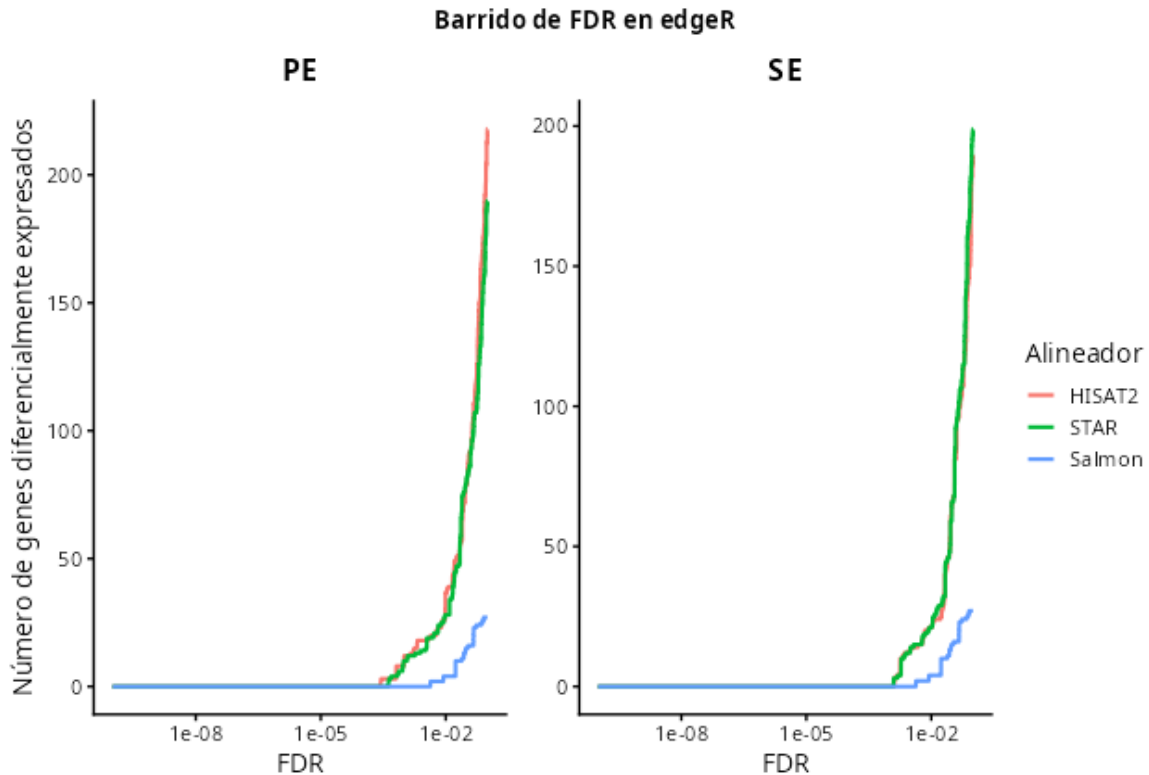


Figura 6: Barrido de valores de FDR y su efecto en el número de genes diferencialmente expresados para el algoritmo EdgeR.

La **Figura 6** exhibió un comportamiento donde la caída en el número de genes fue mucho más pronunciado en valores relativamente altos de FDR. Lo anterior se contrapone a los resultados de ambas implementaciones de DESeq2, las cuales fueron más robustas ante un FDR más estricto.

```
# ===== TABLAS =====

# Directorios
results_dir_pe = list(
  hisat2 = "results/hisat2_pe/diffexp_edgeR",
  star = "results/star_pe/diffexp_edgeR",
  salmon = "results/SALMON/PE/diffexp_edgeR"
)

results_dir_se = list(
  hisat2 = "results/hisat2_se/diffexp_edgeR",
  star = "results/star_se/diffexp_edgeR",
  salmon = "results/SALMON/SE/diffexp_edgeR"
)
```

```

# Parámetros
FDR_unique = 0.05

for (i in 1:2) {
  if (i == 1) {
    res = edger_results_pe
    results_dir = results_dir_pe
  } else {
    res = edger_results_se
    results_dir = results_dir_se
  }
  for (tool in count_tools) {

    # Creación del directorio en caso de no existir
    if (!dir.exists(results_dir[[tool]])) {
      dir.create(results_dir[[tool]], recursive = TRUE)
    }

    # ===== Tablas con FDR y LFC fijos =====

    up = (res[[tool]]$logFC > LFC_unique) &
      (res[[tool]]$FDR < FDR_unique)
    up[is.na(up)] = FALSE

    down = (res[[tool]]$logFC < -LFC_unique) &
      (res[[tool]]$FDR < FDR_unique)
    down[is.na(down)] = FALSE

    # Guardado de tablas UP/DOWN
    write.table(
      res[[tool]][up, ],
      file = file.path(
        results_dir[[tool]],
        paste0("edgeR-DEG-UP-", FDR_unique, ".csv")
      ),
      sep = ",",
      quote = FALSE,
      row.names = TRUE
    )

    write.table(
      res[[tool]][down, ],
      file = file.path(
        results_dir[[tool]],
        paste0("edgeR-DEG-DOWN-", FDR_unique, ".csv")
      ),
      sep = ",",
      quote = FALSE,
      row.names = TRUE
    )
  }
}

```

```
    )  
  }  
}  
  
rm(res, tool)  
invisible(gc())
```

2.8 Volcano plot

En este segmento se generaron *volcano plots*, gráficos útiles para relacionar la significancia estadística (mediante el p-value ajustado) y el LFC. Estos siguieron los umbrales aplicados en la sección pasada:

- $FDR < 0.05$.
- $|LFC| > 0.5$.

2.8.1 DESeq2

Aunado a ello, se guardaron todas las tablas resultantes de la implementación de `inmoose` para ser cargados como objetos de R.

2.8.1.1 R

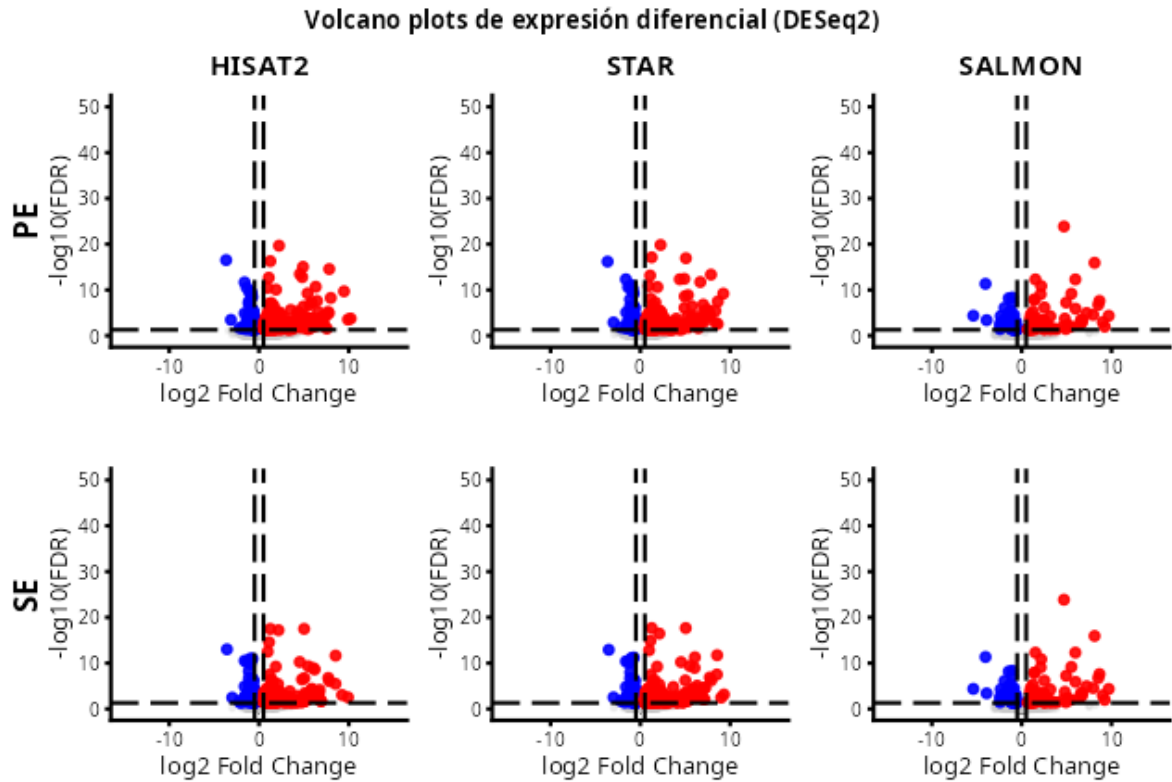


Figura 7: Volcano plots para DESeq2 de todos los datasets (tres alineadores y dos tipos de lectura). Se usó un $FDR < 0.05$, un $LFC > 0.5$ para los up y $LFC < -0.5$ para los down.

Como se evidenció anteriormente, bajo el mismo umbral hay una proporción importante de genes diferencialmente expresados tanto **UP** como **DOWN** ante un FDR laxo.

2.8.1.2 Python

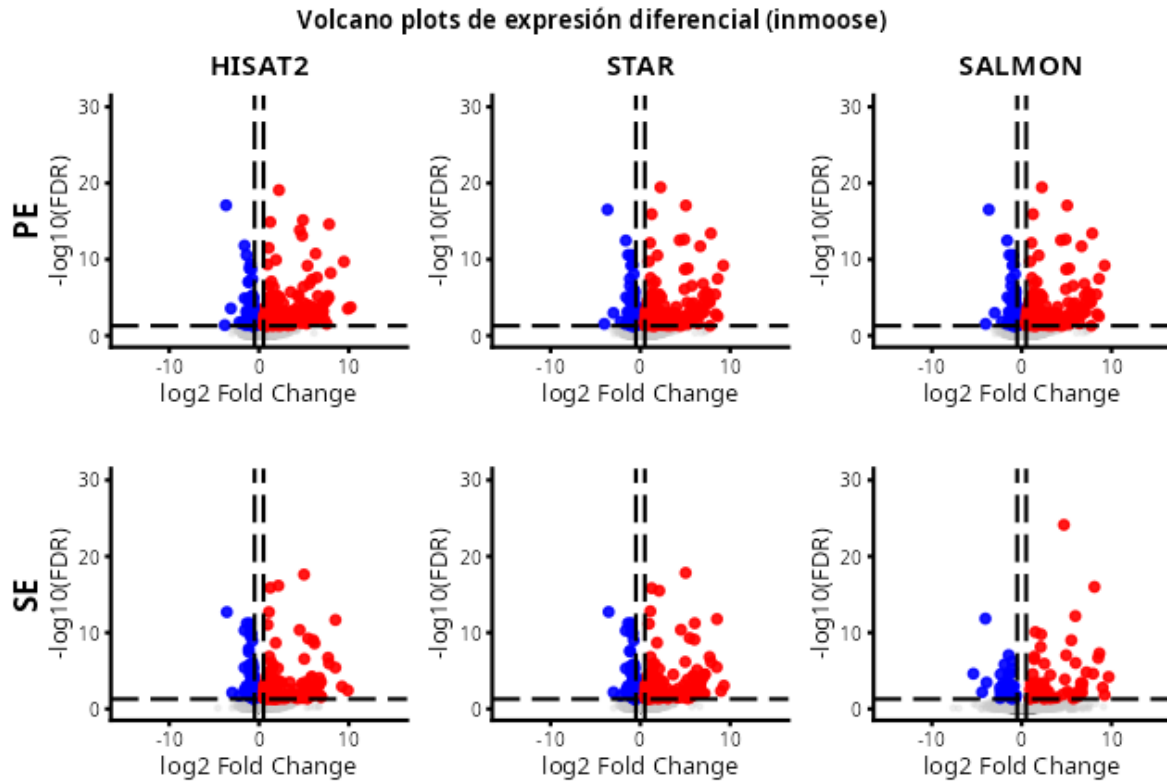


Figura 8: Volcano plots para DESeq2 de la suite *inmoose* de todos los datasets (tres alineadores y dos tipos de lectura). Se usó un $FDR < 0.05$, un $LFC > 0.5$ para los up y $LFC < -0.5$ para los down.

Se mantuvo un patrón similar a su contraparte en R, con los alineadores siguiendo tendencias similares en los gráficos con mayores diferencias en contraste con SALMON.

2.8.2 EdgeR

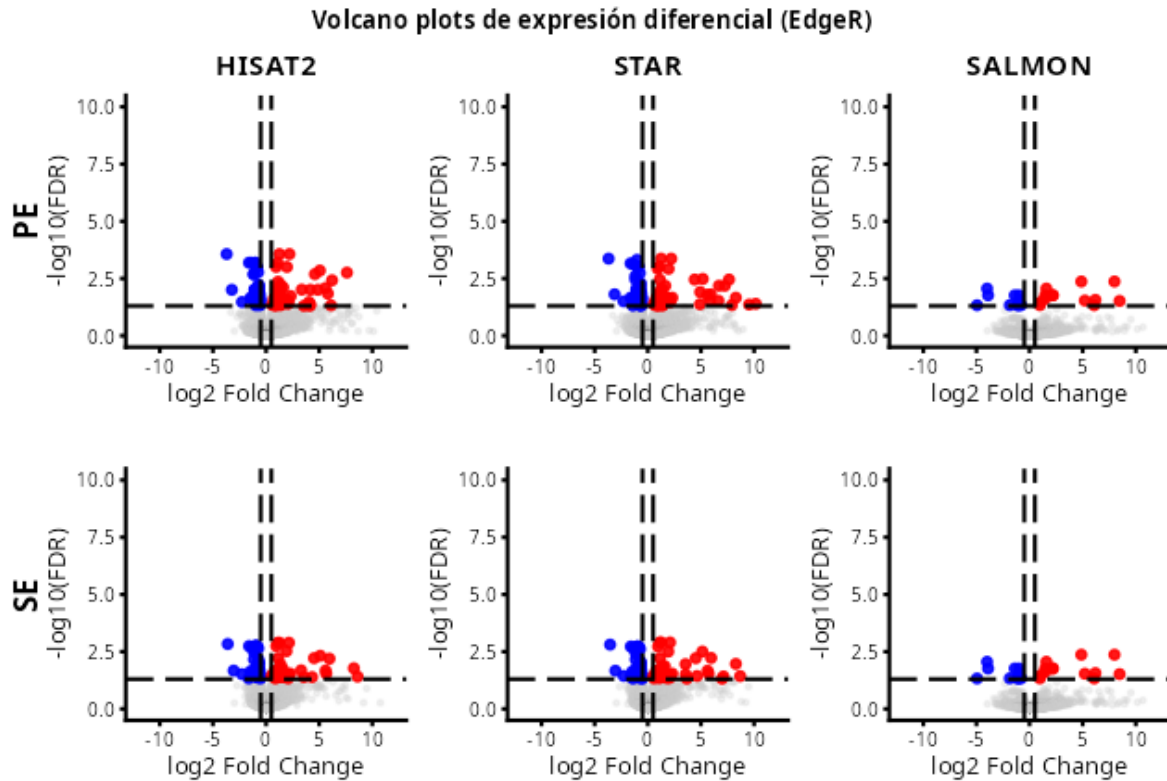


Figura 9: Volcano plots para EdgeR de todos los datasets (tres alineadores y dos tipos de lectura). Se usó un $FDR < 0.05$, un $LFC > 0.5$ para los up y $LFC < -0.5$ para los down.

De nueva cuenta, EdgeR mostró un comportamiento más conservador en todos los datasets con relación a DESeq2.

2.9 Heatmap

Para la construcción de estas representaciones no se trabajó con TPMs, puesto que en pasos previos se generaron matrices transformadas de los conteos que facilitan la comparación entre muestras. Tal fue el caso de los $\log_{cpm}$ de EdgeR y vst de DESeq2.

2.9.1 DESeq2

2.9.1.1 R

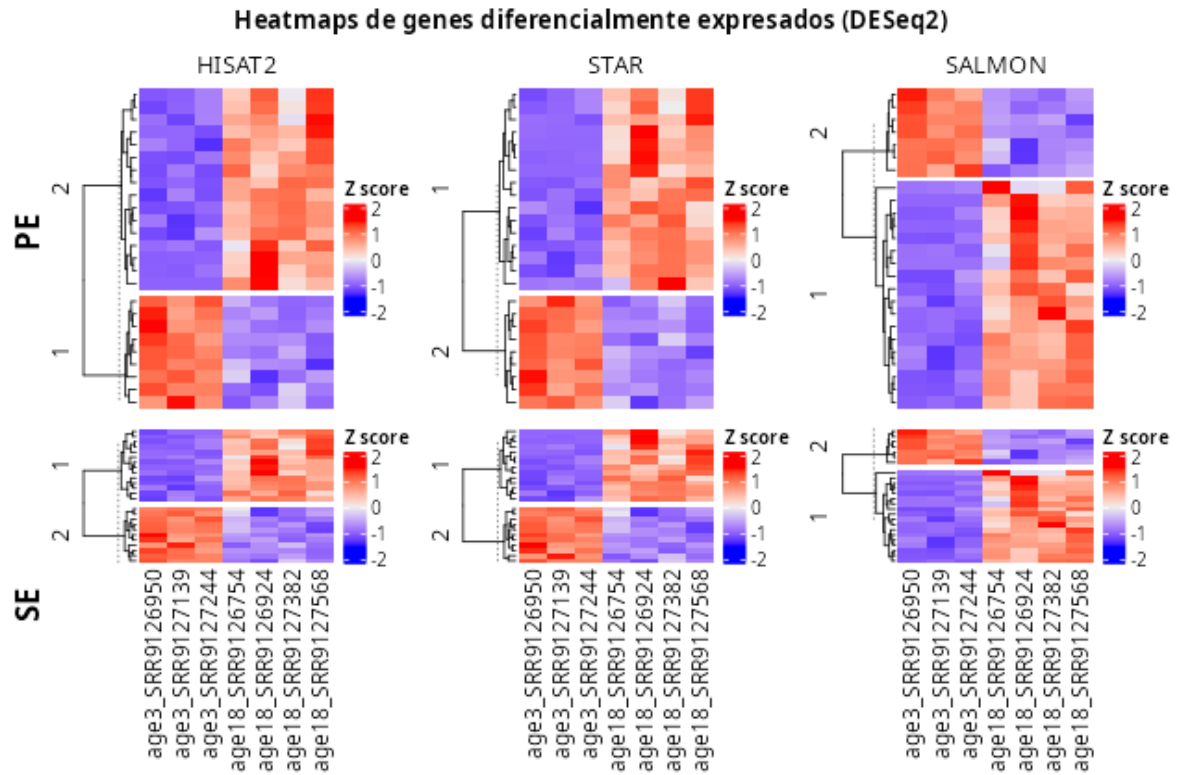


Figura 10: Top 25 genes diferencialmente expresados para cada dataset procesado con el algoritmo DESeq2.

2.9.1.2 Python

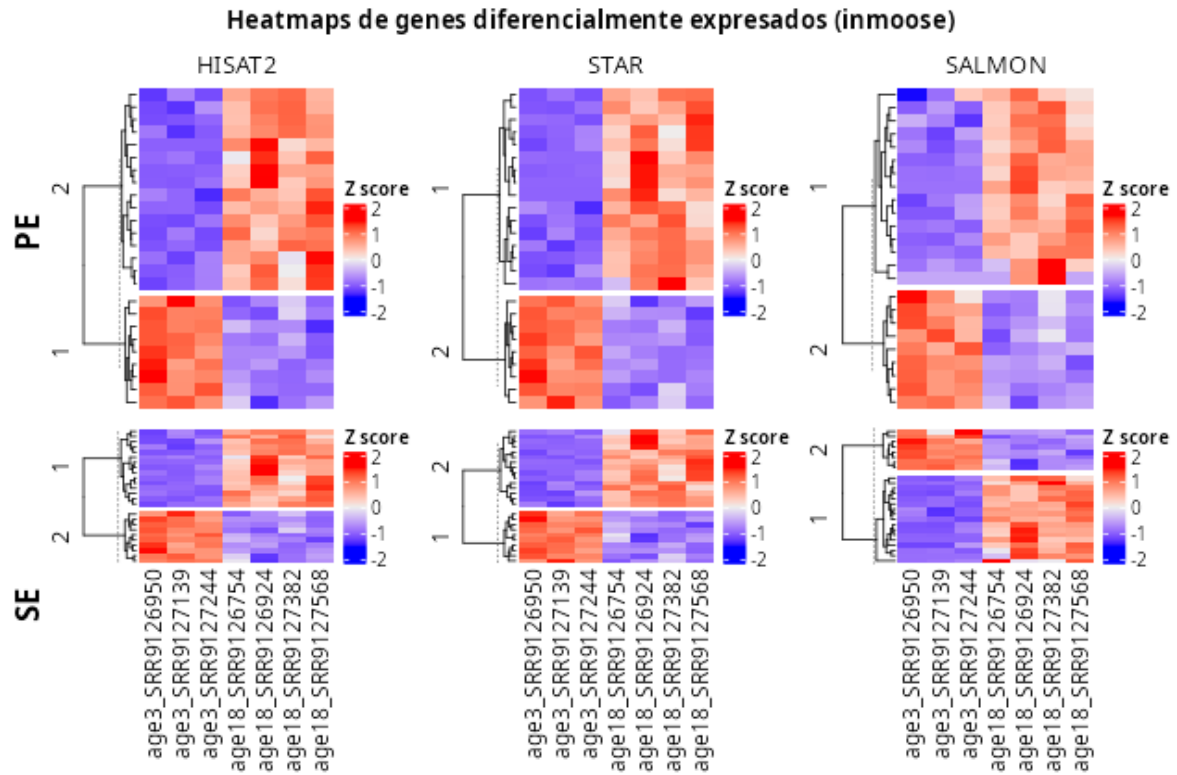


Figura 11: Top 25 genes diferencialmente expresados para cada dataset procesado con el algoritmo DESeq2 en la suite inmoose.

2.9.2 EdgeR

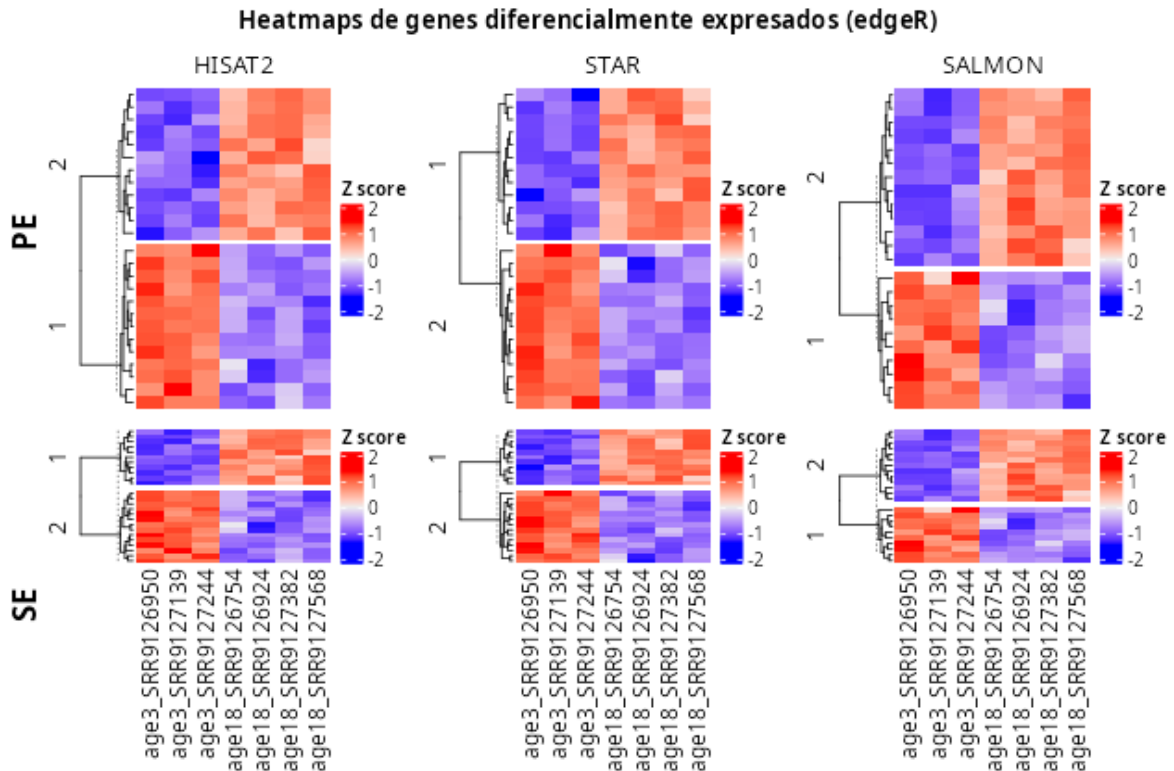


Figura 12: Top 25 genes diferencialmente expresados para cada dataset procesado con el algoritmo EdgeR.

2.10 Intersección de genes entre algoritmos y datasets

En este apartado se buscó la intersección de los genes obtenidos por la clasificación de cada algoritmo (up- o down-regulated) entre datasets (tipo de lectura y alineador/pseudoalineador).

2.10.1 Genes sobre-regulados

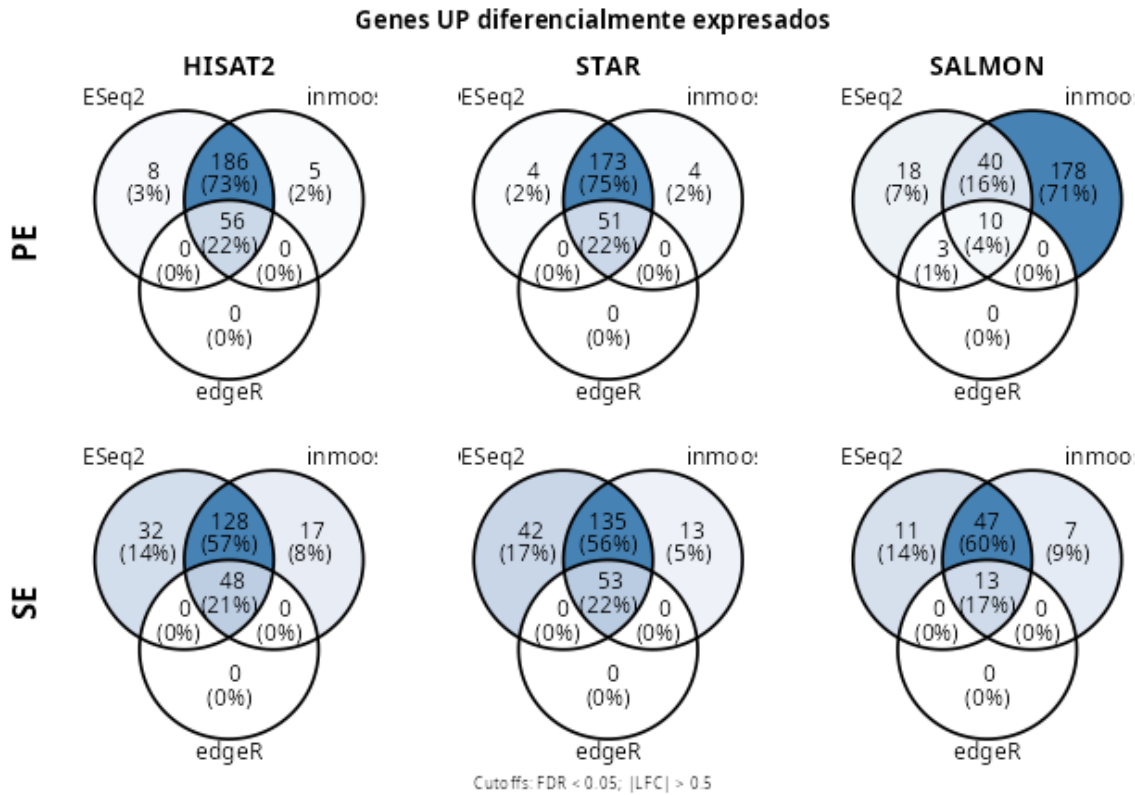


Figura 13: Diagramas de Venn para genes diferencialmente sobreexpresados comparando DESeq2, inmoose y edgeR por alineador y tipo de lectura.

La figura muestra que los genes **up diferencialmente expresados** presentan una alta concordancia entre DESeq2 e inmoose, especialmente en HISAT2 y STAR, tanto para PE como para SE. En cambio, edgeR no comparte genes UP exclusivos ni intersecciones relevantes con los otros métodos, lo que sugiere que bajo este umbral es más conservador o detecta un conjunto muy reducido de genes.

También se observa que **Salmon** tiene un comportamiento distinto: en PE, inmoose detecta una mayor proporción de genes exclusivos, mientras que en SE la intersección entre DESeq2 e inmoose sigue siendo la región dominante. En general, la figura indica que la mayor consistencia ocurre entre las implementaciones tipo DESeq2, mientras que edgeR contribuye poco a la intersección de genes UP con los criterios de ($FDR < 0.05$) y ($|LFC| > 0.5$).

2.10.2 Genes sub-regulados

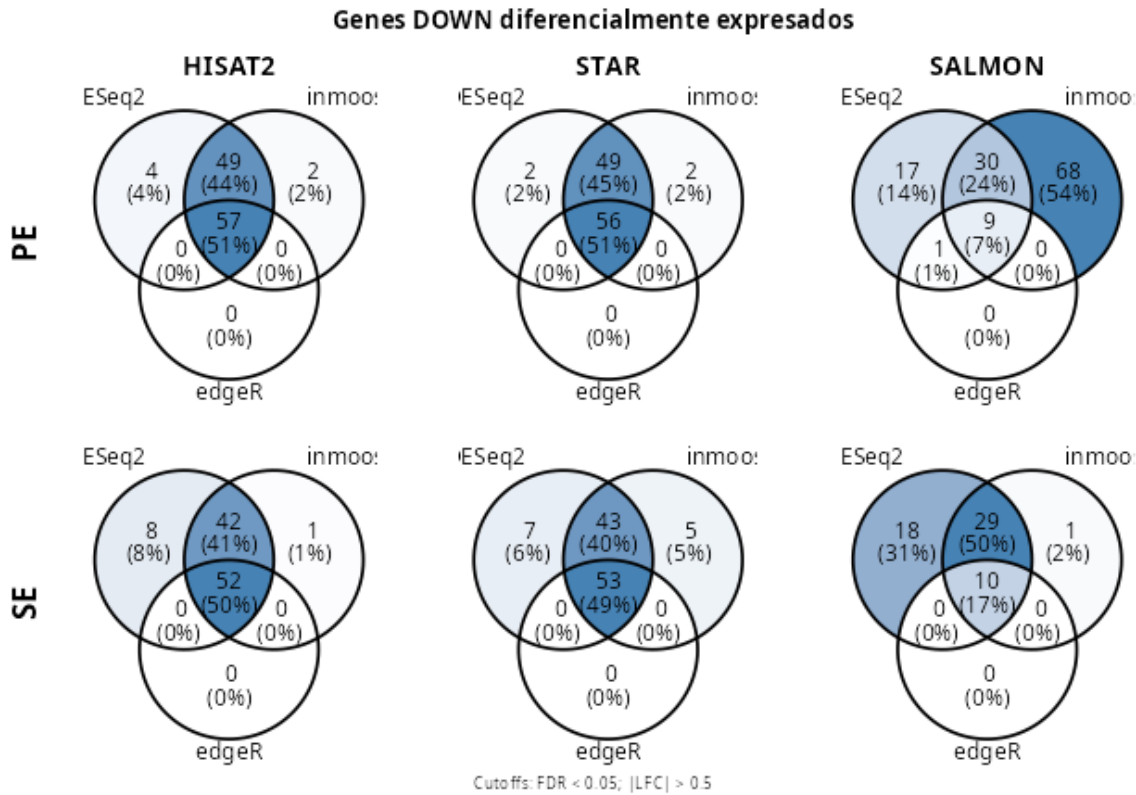


Figura 14: Diagramas de Venn para genes diferencialmente subexpresados comparando DESeq2, inmoose y edgeR por alineador y tipo de lectura.

La figura muestra que los genes **DOWN diferencialmente expresados** mantienen una concordancia alta entre **DESeq2** e **inmoose**, especialmente en HISAT2 y STAR, donde la intersección entre ambos métodos concentra la mayor parte de los genes detectados. En contraste, **edgeR** no aporta genes exclusivos ni intersecciones relevantes, lo que indica que bajo este criterio es el método más restrictivo para genes subexpresados.

También se observa que **Salmon** presenta un patrón más variable: en PE, inmoose detecta más genes exclusivos, mientras que en SE la mayor proporción corresponde a genes compartidos entre DESeq2 e inmoose. En conjunto, la figura sugiere que la señal de genes DOWN es consistente principalmente entre DESeq2 e inmoose, mientras que edgeR contribuye poco a la detección compartida con ($FDR < 0.05$) y ($|LFC| > 0.5$).

3 Discusión

El análisis muestra que la detección de genes diferencialmente expresados depende de manera importante tanto del método estadístico utilizado como del umbral de significancia seleccionado. En particular, se observa que **edgeR presenta un comportamiento más conservador** en comparación con DESeq2 y su implementación en Python mediante inmoose. Esto se refleja en un menor número de genes diferencialmente expresados al aplicar el ajuste por múltiples pruebas mediante FDR, especialmente cuando se utilizan criterios combinados de ($FDR < 0.05$) y ($|LFC| > 0.5$).

Este comportamiento puede explicarse por la forma en que edgeR modela la dispersión y evalúa la significancia estadística. Al utilizar el enfoque de quasi-likelihood con `glmQLFit()` y `glmQLFTest()`, edgeR incorpora una estimación más cuidadosa de la incertidumbre asociada a la dispersión biológica [1]. En consecuencia, tiende a ser más estricto al declarar genes como significativos, lo que reduce la probabilidad de cometer errores de tipo I, es decir, falsos positivos. Bajo esta lógica, edgeR sacrifica sensibilidad para ganar mayor control sobre la detección espuria de genes diferencialmente expresados.

En contraste, DESeq2 en R e inmoose en Python muestran una alta convergencia entre sí. Esto se observa tanto en los diagramas de Venn como en los volcano plots y heatmaps, donde ambos métodos tienden a recuperar conjuntos de genes muy similares, especialmente para los resultados provenientes de HISAT2 y STAR. Esta concordancia sugiere que la implementación de inmoose reproduce de forma consistente la lógica estadística central de DESeq2, al menos bajo las condiciones evaluadas en este trabajo. Por ello, una hipótesis razonable es que las diferencias menores entre ambas implementaciones pueden deberse más a detalles computacionales, redondeo, filtrado previo, manejo de valores faltantes o diferencias en las transformaciones internas que a discrepancias metodológicas profundas.

También se observa que el impacto del FDR seleccionado es determinante para la cantidad de genes recuperados. El barrido de FDR permite ver que pequeños cambios en el umbral pueden modificar de forma considerable el número de genes diferencialmente expresados. Esto ocurre porque el FDR controla la proporción esperada de falsos descubrimientos entre los genes declarados significativos; por lo tanto, un umbral más estricto, como (1×0.05), selecciona únicamente genes con evidencia estadística muy fuerte, mientras que un umbral más flexible, como (0.05), permite recuperar un conjunto más amplio de genes candidatos.

Esta diferencia es relevante porque el FDR no solo afecta la cantidad de genes obtenidos, sino también la interpretación biológica posterior. Un umbral muy estricto puede reducir falsos positivos, pero también puede excluir genes biológicamente relevantes con efectos moderados. Por otro lado, un umbral más permisivo aumenta la sensibilidad, pero también puede introducir genes menos confiables. Por ello, el análisis no debe depender únicamente de un punto de corte arbitrario, sino que debe considerar cómo cambia la señal diferencial a lo largo del barrido de FDR.

En conjunto, los resultados sugieren que DESeq2 e inmoose ofrecen una **detección más concorde y posiblemente más sensible bajo estos datos**, mientras que edgeR actúa como un método más conservador frente al ajuste por múltiples comparaciones. Una hipótesis general es que la estructura del experimento, el tamaño muestral, la dispersión biológica y las diferencias entre conteos derivados de alineamiento o pseudoalineamiento influyen directamente en el comportamiento de cada método. Por ello, comparar varios enfoques permite distinguir genes robustos, detectados de manera consistente entre herramientas, de genes cuya significancia depende más fuertemente del modelo estadístico o del umbral de FDR utilizado.

4 Conclusiones

Con base en la comparación realizada, se selecciona el dataset **paired-end procesado con STAR** y el análisis diferencial con **DESeq2 en cualquier implementación** como estrategia principal. Esta elección se justifica porque STAR-PE mostró una señal diferencial clara, consistente y comparable con HISAT2, pero con una buena recuperación de genes tanto en los volcano plots como en los barridos de FDR. Además, los resultados de DESeq2 en R convergieron ampliamente con los obtenidos mediante **inmoose**, lo que refuerza la reproducibilidad del análisis entre implementaciones.

Aunque **edgeR** mostró un comportamiento más conservador frente al ajuste por FDR, su baja recuperación de genes sugiere que podría utilizarse mejor como análisis de contraste o validación estricta, más que como herramienta principal en este caso. Por otro lado, **Salmon** presentó patrones más variables, especialmente en las intersecciones de genes UP y DOWN, por lo que se considera menos estable para tomarlo como dataset principal.

En conclusión, STAR **paired-end + DESeq2** representa el balance más adecuado entre sensibilidad, consistencia metodológica y claridad interpretativa para este análisis de expresión diferencial.

Referencias

1. Yunshun Chen, Lizhong Chen, Aaron T. L. Lun, Pedro L. Baldoni, Gordon K. Smyth. **edgeR v4: powerful differential analysis of sequencing data with expanded functionality and improved support for small counts and larger datasets**. *Nucleic Acids Research*. 2025 Jan;53(2). doi:[10.1093/nar/gkaf018](https://doi.org/10.1093/nar/gkaf018)
2. Simon Anders, Wolfgang Huber. **Differential expression analysis for sequence count data**. *Genome Biology*. 2010 Oct;11(10):R106. doi:[10.1186/gb-2010-11-10-r106](https://doi.org/10.1186/gb-2010-11-10-r106)
3. Maximilien Colange, Guillaume Appé, Léa Meunier, Solène Weill, Akpéli Nordor, Abdelkader Behdenna. **Differential expression analysis with inmoose, the integrated multi-omic open-source environment in Python**. *BMC Bioinformatics*. 2025 Jun;26(1):160. doi:[10.1186/s12859-025-06180-7](https://doi.org/10.1186/s12859-025-06180-7)
4. David Valle-Garcia. **Clase 8 - Análisis de expresión diferencial con DESeq2**. 2026.
5. Charity W. Law, Kathleen Zeglinski, Xueyi Dong, Monther Alhamdoosh, Gordon K. Smyth, Matthew E. Ritchie. **A guide to creating design matrices for gene expression experiments [Internet]**. 2020. Available from: <https://bioconductor.org/packages/release/workflows/vignettes/RNAseq123/inst/doc/designmatrices.html>
6. Jesse Perla, Thomas J. Sargent, John Stachurski. **Orthogonal Projections and Their Applications [Internet]**. *QuantEcon*; 2024. Available from: https://julia.quantecon.org/tools_and_techniques/orth_proj.html